CS 550 - Machine Learning
Homework Assignment #3

Contact: a.yildirim@bilkent.edu.tr

Deadline: 02.05.2025 - 23:59

In this homework, you are expected to implement different artificial neural network (ANN) models from scratch and train them using the backpropagation algorithm. You will analyze how these models perform on functions with varying levels of complexity and the effect of the hyperparameters on their prediction performance.

The use of machine learning libraries (e.g., scikit-learn, TensorFlow, or PyTorch) is strictly prohibited. However, you may use libraries such as NumPy and Pandas for data manipulation and matrix operations. You may use any programming language as long as your implementation complies with these requirements.

1 Dataset Analysis

The assignment provides three separate datasets named according to the complexity of the underlying function that the model is expected to approximate: `easy`, `medium`, and `hard`. Each dataset is divided into two files, one for training and one for testing, resulting in six text files in total:

- `easy_train.txt`, `easy_test.txt`
- `medium_train.txt`, `medium_test.txt`
- `hard_train.txt`, `hard_test.txt`

Each file contains a sequence of input-output pairs, where both the input and output are scalar (i.e., one-dimensional values). Each line in the file corresponds to a single data point, formatted as:

```
x y
```

These datasets vary in terms of function complexity and smoothness, which will influence both the model choice (linear vs. nonlinear) and the effectiveness of different hyperparameter configurations. A well-tuned simple model may

perform adequately on the `easy` dataset, while the `hard` dataset may require a more complex model.

As a first step, you are requested to visualize these datasets on a two-dimensional plot, include them in your report, and discuss their difference in terms of learning complexity.

2 Model Implementation

In this part of the homework, you will implement two artificial neural networks (ANN) models: a Single-Layer Perceptron (SLP), which performs linear regression, and a Multi-Layer Perceptron (MLP), which includes a single hidden layer. Both models should not have any activation function in the output layer as the task is regression. They must be built entirely from scratch and should support a set of hyperparameter configuration options, which can later be tuned during training.

Model weights should be initialized by sampling independently from a uniform distribution within the following range:

$$w \sim \mathcal{U}(-1, 1)$$

where -1 and 1 are the lower and upper bounds of the range. This initialization applies to all layers. Also, all biases in the network should be initialized to zero:

$$b = 0$$

This approach ensures that the initial outputs are unbiased while still allowing gradient-driven updates during training.

The models are expected to support the following parameters:

- **Learning rate:** Controls the step size for parameter updates during optimization.
- **Hidden layer size (MLP only):** Determines the number of neurons in the hidden layer.
- **Activation function (MLP only):** Non-linear functions applied to hidden layer outputs. The following are required to be supported:
  - **Tanh:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
  - **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$
  - **ReLU:** $\text{ReLU}(x) = \max(0, x)$
- **Momentum:** A mechanism to incorporate the history of gradients into the current update, improving convergence stability. The following formulation is used:

  $$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla J(\theta), \quad \theta \leftarrow \theta - \eta \cdot v_t$$

  where θ denotes model parameters, η is the learning rate, J is the loss function, and β is the momentum coefficient. When momentum is not used, standard gradient descent is applied.

Each model must implement a set of functions to support training from scratch. The **constructor** is responsible for initializing all model parameters, including the learning rate, loss type, momentum coefficient, and weight initialization range. In the case of the MLP model, the activation function and the size of the hidden layer must also be specified. During training, the **forward pass** computes the model's output for a given batch of inputs. The **backpropagation** function performs a single training step by computing gradients for all weights and biases, applying the **MSE loss function**, and updating parameters accordingly. If momentum is enabled, it must be incorporated into the parameter updates to help stabilize and accelerate convergence. Additionally, the MLP must handle the computation of activation function derivatives during backpropagation to properly update the hidden layer.

In your report, clearly describe the implementation of your models and the configurable parameters they support. Explain each parameter's purpose by including their possible effects in the training process.

These models will serve as the foundation for all training and evaluation steps in the remaining parts of the assignment.

3 Analysis of Hyperparameters

In this section, you will investigate how the implemented ANN models perform under various configurations and on datasets of different complexity. Your analysis should aim to provide insights into the learning capabilities and sensitivity of the models to hyperparameter choices. **In this part of the assignment, min-max scaling should not be applied to the dataset.**

Evaluate how well each model (SLP and MLP) fits the `easy`, `medium`, and `hard` datasets using the Mean Squared Error (MSE) metric. In the training setup, use **MSE loss** and **set the number of epochs to** 2000 **and the mini-batch size to** 16. Continue training until the epoch number is reached. You are requested to conduct experiments by trying combinations of the following hyperparameters:

- **Learning Rate:** {0.01, 0.1, 1 }
- **Momentum Coefficient:** {No momentum, 0.5, 0.9}
- **Hidden Layer Size (MLP only):** {4, 8, 16}
- **Activation Function (MLP only):** {sigmoid, tanh, relu}

For each dataset, explore the interaction between these settings and analyze their influence on training dynamics and final performance. In your report, provide:

- Find the best SLP and best MLP model per dataset based on their MSE scores on the test datasets (easy, medium, and hard). If the model cannot

converge, consider the first configuration to be the best model. Report their MSE score and hyperparameter configurations on a table.

- By uniformly sampling x values between the range of ground-truth values, visualize the predicted function of each best-performing model on the test and training datasets. Include ground-truth values in the plot to compare the prediction results.
- Discuss how hyperparameter selection is affected by the complexity of the datasets. **Could there be better predictions for them? Explain why.**
- Discuss the difference between SLP and MLP models in terms of learning capability.

4 Min-Max Scaling

In this section, you will investigate the effect of applying min-max normalization to both the input and output values. **On the `medium` dataset**, repeat the same hyperparameter search described in Part 3 using the normalized data. Min-max values should be extracted from the training data, and the following equation should be applied for normalization of both test and training data:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad y_{\text{scaled}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$$

Report the best-performing model configurations (both SLP and MLP) and their corresponding MSE scores on the training and test sets.

After completing the experiments, compare the results to those obtained without min-max scaling. Discuss whether normalization improves performance, worsens it, or has minimal effect. **In your report, discuss why this might be the case considering your dataset properties. Comparing to Part 3 results, discuss the difference between SLP and MLP models in terms of learning capability. Did your observations change?**

5 Effect of Model Complexity

In this experiment, you will analyze the effect of model complexity on all three datasets: `easy`, `medium`, and `hard`. The goal is to understand how the number of hidden units in the MLP model influences its ability to approximate functions of varying complexity.

For each dataset, begin by selecting the best-performing MLP configuration identified in the previous experiments (including those that used min-max scaling). Using these configurations as a baseline, train new models by varying the hidden layer size: 2, 4, 8, and 16 neurons. **Set the number of epochs to** 5000 and keep the rest of the training setups the same as in Part 3.

For each dataset and each hidden layer size, visualize the prediction performance as in Part 3.

Compare the prediction results across different hidden sizes and datasets. **Discuss how increasing the model's capacity affects its ability to fit the training data and generalize to the test set.** Reflect on whether larger models consistently perform better or if they lead to overfitting or unnecessary complexity.

6 Submission

- You are required to submit a report of maximum **5 pages**, prepared using the official IEEE conference template.
  - The quality and clarity of your tables and figures will contribute to your grade.
  - Do not include any screenshots in your report.
  - Make sure all questions and instructions in each section are fully addressed.
  - While five pages is the upper limit, shorter reports that thoroughly answer all required questions will not be penalized.
- Submit your implementation code as a separate file alongside the report.